

Experiment 2

Academic Year 2026-2027			
Program: Electronics Engineering (VLSI Design and Technology)			
Year of Study	Second Year	Semester	III
Course	Skill Lab: Python Programming	Course Code	VDCOR2VS201
Class	SY VLSI	Course In-Charge	Mr. Nirmol Munvar
Exp Name	List, Tuples and Sets		
CO	Apply fundamental Python programming concepts such as variables, data types, operators, and input-output statements to develop basic programs for solving computational problems.		

Lists

A **list** is an ordered collection of items.

Lists are written using square brackets [].

```
numbers = [10, 20, 30, 40]
```

A list can contain different types of data:

```
data = [10, "Python", 3.14, True]
```

Important properties of lists

Lists are:

- **Ordered** — items have positions/indexes.
- **Mutable** — you can change their contents.
- **Allow duplicates.**
- Can contain different data types.

Example:

```
fruits = ["apple", "banana", "mango", "apple"]
```

```
print(fruits)
```

Output:

```
['apple', 'banana', 'mango', 'apple']
```

Notice that "apple" can appear more than once.

List Indexing

Each item in a list has an index.

Python uses **zero-based indexing**.

```
fruits = ["apple", "banana", "mango", "orange"]
```

The positions are:

```
Index:  0      1      2      3
        ↓      ↓      ↓      ↓
      apple banana mango  orange
```

So:

```
print(fruits[0])
```

```
print(fruits[2])
```

Output:

```
apple
```

```
mango
```

Negative indexing

Python also allows negative indexes.

Index:	-4	-3	-2	-1
	↓	↓	↓	↓
	apple	banana	mango	orange

Therefore:

```
print(fruits[-1])
```

Output:

orange

And:

```
print(fruits[-2])
```

Output:

mango

Lists Are Mutable

Mutable means that an object's contents can be changed after the object has been created.

For example:

```
numbers = [10, 20, 30]
```

```
numbers[1] = 200
```

```
print(numbers)
```

Output:

```
[10, 200, 30]
```

We changed 20 to 200.

This is what we mean when we say:

Lists are mutable.

Common List Methods

Python provides many methods for working with lists.

append()

Adds an item to the end.

```
numbers = [10, 20, 30]
```

```
numbers.append(40)
```

```
print(numbers)
```

Output:

```
[10, 20, 30, 40]
```

insert()

Adds an item at a specific position.

```
numbers = [10, 20, 30]
```

```
numbers.insert(1, 15)
```

```
print(numbers)
```

Output:

```
[10, 15, 20, 30]
```

remove()

Removes the first matching value.

```
numbers = [10, 20, 30, 20]
```

```
numbers.remove(20)
```

```
print(numbers)
```

Output:

```
[10, 30, 20]
```

Only the **first** 20 was removed.

pop()

Removes and returns an item.

```
numbers = [10, 20, 30]
```

```
x = numbers.pop()
```

```
print(x)
```

```
print(numbers)
```

Output:

```
30
```

```
[10, 20]
```

You can also specify an index:

```
numbers = [10, 20, 30]
```

```
numbers.pop(1)
```

```
print(numbers)
```

Output:

```
[10, 30]
```

sort()

Sorts a list.

```
numbers = [40, 10, 30, 20]
```

```
numbers.sort()
```

```
print(numbers)
```

Output:

```
[10, 20, 30, 40]
```

Reverse sorting:

```
numbers.sort(reverse=True)
```

reverse()

Reverses the order of the list.

```
numbers = [10, 20, 30]
```

```
numbers.reverse()
```

```
print(numbers)
```

Output:

```
[30, 20, 10]
```

count()

Counts how many times a value occurs.

```
numbers = [10, 20, 20, 30, 20]
```

```
print(numbers.count(20))
```

Output:

3

index()

Finds the index of the first occurrence.

```
fruits = ["apple", "banana", "mango"]
```

```
print(fruits.index("banana"))
```

Output:

1

Useful Built-in Functions with Lists

Some operations are functions rather than list methods.

```
numbers = [10, 20, 30, 40]
```

len()

```
print(len(numbers))
```

Output:

4

sum()

```
print(sum(numbers))
```

Output:

100

min() and max()

```
print(min(numbers))
```

```
print(max(numbers))
```

Output:

10

40

Slicing

Slicing allows you to extract part of a sequence.

The basic syntax is:

```
sequence[start:stop]
```

Important:

start is included, but stop is excluded.

For example:

```
numbers = [10, 20, 30, 40, 50]
```

```
print(numbers[1:4])
```

Output:

```
[20, 30, 40]
```

The indexes are:

	10	20	30	40	50
Index:	0	1	2	3	4
		↑		↑	
		start		stop	

The element at index 4 is not included.

Slicing from the beginning

```
numbers[:3]
```

means:

Start from the beginning and stop before index 3.

Result:

```
[10, 20, 30]
```

Slicing until the end

```
numbers[2:]
```

Result:

```
[30, 40, 50]
```

Slicing with a step

The full syntax is:

```
sequence[start:stop:step]
```

Example:

```
numbers = [0, 1, 2, 3, 4, 5, 6]
```

```
print(numbers[1:6:2])
```

Output:

```
[1, 3, 5]
```

It takes every second element.

Reverse using slicing

A step of -1 goes backwards:

```
numbers = [1, 2, 3, 4, 5]
```

```
print(numbers[::-1])
```

Output:

```
[5, 4, 3, 2, 1]
```

Tuples

A **tuple** is another type of sequence in Python.

Tuples are usually written using parentheses:

```
numbers = (10, 20, 30)
```

Like lists, tuples:

- Are ordered.
- Have indexes.
- Allow duplicates.
- Can contain different data types.
- Support slicing.

But there is one major difference:

Tuples are immutable.

Tuple Indexing

```
colors = ("red", "green", "blue")
```

```
print(colors[0])
```

```
print(colors[2])
```

Output:

```
red
```

```
blue
```

Negative indexing works too:

```
print(colors[-1])
```

Output:

```
blue
```

Tuple Immutability

Consider:

```
numbers = (10, 20, 30)
```

This will cause an error:

```
numbers[1] = 200
```

Why?

Because tuples are **immutable**.

Once the tuple has been created, its elements cannot be changed, added, or removed.

Compare:

```
numbers = [10, 20, 30]
```

```
numbers[1] = 200
```

This is allowed because lists are mutable.

But:

```
numbers = (10, 20, 30)
```

```
numbers[1] = 200
```

is not allowed because tuples are immutable.

Why Use Tuples?

Tuples are useful when you have a collection of values that shouldn't be changed.

For example:

```
coordinates = (10, 20)
```

You might represent:

```
x = 10
```

```
y = 20
```

Another example:

```
student = ("Rahul", 20, 85.5)
```

If these values are intended to remain fixed, a tuple can be appropriate.

Tuple Methods

Tuples have fewer methods because they cannot be modified.

count()

```
numbers = (10, 20, 20, 30)
```

```
print(numbers.count(20))
```

Output:

```
2
```

index()

```
numbers = (10, 20, 30)
```

```
print(numbers.index(20))
```

Output:

```
1
```

Tuples also support:

```
len()
```

```
min()
```

```
max()
```

```
sum()
```

when the contents are appropriate.

Tuple Slicing

Tuples support slicing just like lists.

```
numbers = (10, 20, 30, 40, 50)
```

```
print(numbers[1:4])
```

Output:

```
(20, 30, 40)
```

And:

```
print(numbers[::-1])
```

Output:

```
(50, 40, 30, 20, 10)
```

Sets

A **set** is a collection of unique elements.

Sets are written using curly brackets:

```
numbers = {10, 20, 30}
```

The most important property is:

A set does not contain duplicate elements.

For example:

```
numbers = {10, 20, 20, 30, 30, 30}
```

```
print(numbers)
```

The result will contain only unique values:

```
{10, 20, 30}
```

Sets Are Unordered

Unlike lists and tuples, sets do not provide positional indexing.

This is not valid:

```
numbers = {10, 20, 30}
```

```
print(numbers[0])
```

You cannot ask for "the first element" of a set by index.

A set is primarily useful when you care about **unique values** and **set operations**, rather than positions.

Sets Are Mutable

A normal set is mutable.

You can add and remove elements.

add()

```
numbers = {10, 20, 30}
```

```
numbers.add(40)
```

```
print(numbers)
```

The set now contains 40.

remove()

```
numbers.remove(20)
```

discard()

```
numbers.discard(20)
```

`discard()` is useful because it does not raise an error if the element isn't present.

Set Operations

Sets are particularly useful for mathematical set operations.

Suppose:

```
A = {1, 2, 3, 4}
```

```
B = {3, 4, 5, 6}
```

Union

Everything from both sets:

```
print(A | B)
```

Result:

```
{1, 2, 3, 4, 5, 6}
```

You can also use:

```
A.union(B)
```

Intersection

Elements common to both:

```
print(A & B)
```

Result:

```
{3, 4}
```

Or:

```
A.intersection(B)
```

Difference

Elements in A but not in B:

```
print(A - B)
```

Result:

```
{1, 2}
```

Symmetric Difference

Elements that are in either set, but not both:

```
print(A ^ B)
```

Result:

```
{1, 2, 5, 6}
```

Strings Are Also Sequences

This is an important concept.

A **string** is a sequence of characters.

```
name = "Python"
```

You can think of it as:

```
P y t h o n
```

```
0 1 2 3 4 5
```

Therefore, strings support **indexing**:

```
print(name[0])
```

```
print(name[3])
```

Output:

```
P
```

```
h
```

Negative indexing works too:

```
print(name[-1])
```

Output:

```
n
```

Strings Support Slicing

Because strings are sequences, you can slice them.

```
text = "Python"
```

```
print(text[0:3])
```

Output:

```
Pyt
```

Other examples:

```
print(text[:3])
```

```
print(text[2:])
```

```
print(text[::-1])
```

Output:

```
Pyt
```

```
thon
```

nohtyP

So slicing works with:

- Lists
- Tuples
- Strings

Strings Are Immutable

This is a very important concept.

You **cannot change an individual character** in a string.

For example:

```
name = "Python"
```

```
name[0] = "J"
```

This produces an error.

Why?

Because strings are **immutable**.

You cannot modify the existing string object character-by-character.

Instead, you create a **new string**.

For example:

```
name = "Python"
```

```
name = "J" + name[1:]
```

```
print(name)
```

Output:

Jython

We didn't modify the original string. We created a new string and made name refer to it.

Mutable vs Immutable

This is one of the most important Python concepts.

Mutable

The object's contents can be changed.

Examples:

list

set

Example:

```
numbers = [10, 20, 30]
```

```
numbers[0] = 100
```

```
print(numbers)
```

Output:

```
[100, 20, 30]
```

Immutable

The object's contents cannot be changed after creation.

Examples:

int

float

bool

str

tuple

For example:

```
x = 10
```

You cannot modify the integer object 10.

When you do:

```
x = 20
```

you are making x refer to another integer object.

Similarly:

```
text = "Hello"
```

```
text = "World"
```

doesn't modify "Hello".

It makes text refer to "World".

A Simple Way to Remember Mutability

Think of a **list as a whiteboard**.

You can erase and change what's written on it:

```
[10, 20, 30]
```

↓

```
[10, 200, 30]
```

Think of a **tuple/string as something written in permanent ink**.

You cannot modify an individual part of it.

You have to create a new object instead.

String Methods

Strings have many useful methods.

```
text = "hello python"
```

upper()

```
print(text.upper())
```

Output:

```
HELLO PYTHON
```

lower()

```
print(text.lower())
```

capitalize()

```
print(text.capitalize())
```

Output:

```
Hello python
```

title()

```
print(text.title())
```

Output:

```
Hello Python
```

replace()

```
text = "I like Java"
```

```
print(text.replace("Java", "Python"))
```

Output:

```
I like Python
```

Remember: because strings are immutable, `replace()` creates and returns a new string.

find()

```
text = "Hello Python"
```

```
print(text.find("Python"))
```

Output:

```
6
```

count()

```
text = "banana"
```

```
print(text.count("a"))
```

Output:

3

strip()

Removes whitespace from the beginning and end:

```
text = " Hello "
```

```
print(text.strip())
```

Output:

Hello

The in Operator

The in operator checks whether an item exists in a sequence or collection.

String

```
text = "Python"
```

```
print("P" in text)
```

Output:

True

List

```
numbers = [10, 20, 30]
```

```
print(20 in numbers)
```

Output:

True

Tuple

```
numbers = (10, 20, 30)
```

```
print(50 in numbers)
```

Output:

False

Set

```
numbers = {10, 20, 30}
```

```
print(20 in numbers)
```

Output:

True

Comparing List, Tuple, Set and String

Feature	List	Tuple	Set	String
Ordered	Yes	Yes	No	positional order
Mutable	Yes	No	Yes	No
Duplicates	Yes	Yes	No	Yes
Indexing	Yes	Yes	No	Yes
Slicing	Yes	Yes	No	Yes
Written as	[]	()	{}	" "

A useful mental model:

LIST

Ordered + Mutable + Duplicates allowed

TUPLE

Ordered + Immutable + Duplicates allowed

SET

Unique elements + Mutable + No indexing

STRING

Ordered characters + Immutable

One Important Difference: {}

Be careful with curly brackets.

This:

```
x = {}
```

creates an **empty dictionary**, not an empty set.

To create an empty set:

```
x = set()
```

For example:

```
numbers = set()
```

```
numbers.add(10)
```

```
numbers.add(20)
```

```
print(numbers)
```

Practice Exercises

List Basics

Create a list containing five numbers.

Print:

- The first element
- The last element
- The length of the list
- The largest number
- The smallest number

Modify a List

Given:

```
numbers = [10, 20, 30, 40, 50]
```

Do the following:

- Change 20 to 200.
- Add 60 to the end.
- Remove 30.
- Print the final list.

Expected result:

```
[10, 200, 40, 50, 60]
```

List Slicing

Given:

```
numbers = [10, 20, 30, 40, 50, 60, 70]
```

Print:

- [20, 30, 40]
- [50, 60, 70]
- Every second element

- The list in reverse order

Tuple

Create:

```
student = ("Rahul", 20, 85.5)
```

Print:

- Student's name
- Student's age
- Student's marks
- The type of student

Then try:

```
student[1] = 21
```

Observe the error and explain **why it occurs**.

Set

Given:

```
numbers = {10, 20, 20, 30, 30, 30, 40}
```

Print the set.

Then:

- Add 50.
- Remove 20.
- Check whether 30 exists.
- Check its length.

Set Operations

Given:

```
A = {1, 2, 3, 4, 5}
```

```
B = {4, 5, 6, 7, 8}
```

Find:

- Union
- Intersection
- A - B
- B - A
- Symmetric difference

String Indexing and Slicing

Given:

```
text = "Programming"
```

Print:

- First character
- Last character
- First three characters
- Last three characters
- Characters from index 2 to 6
- The string in reverse

String Methods

Given:

```
text = " python programming "
```

Perform:

- Remove the extra spaces.
- Convert it to uppercase.
- Convert it to title case.
- Replace "python" with "Java".
- Count how many times "p" occurs.

Final Challenge

Write a program that asks the user for **5 numbers** and stores them in a list.

Then:

- Print the list.
- Print its length.
- Print the largest number.
- Print the smallest number.
- Print the sum.
- Print the list in reverse using **slicing**.
- Convert the list to a set and print it.
- Print the type of the list.
- Print the type of the set.

For example:

Enter number 1: 10

Enter number 2: 20

Enter number 3: 20

Enter number 4: 30

Enter number 5: 40

You should end up with something like:

List: [10, 20, 20, 30, 40]

Length: 5

Maximum: 40

Minimum: 10

Sum: 120

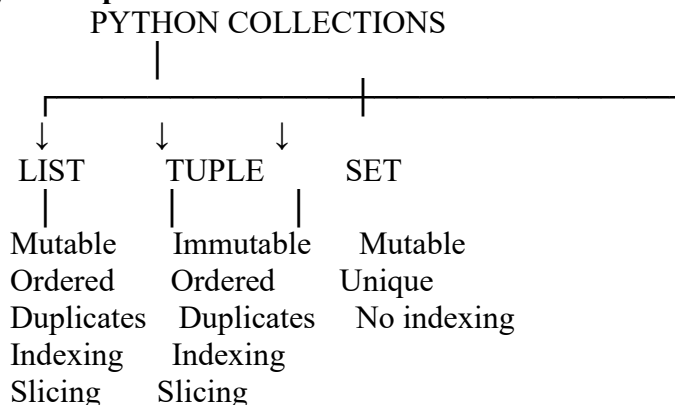
Reverse: [40, 30, 20, 20, 10]

Set: {10, 20, 30, 40}

List type: <class 'list'>

Set type: <class 'set'>

Key Concepts to Master



And remember:

List → mutable

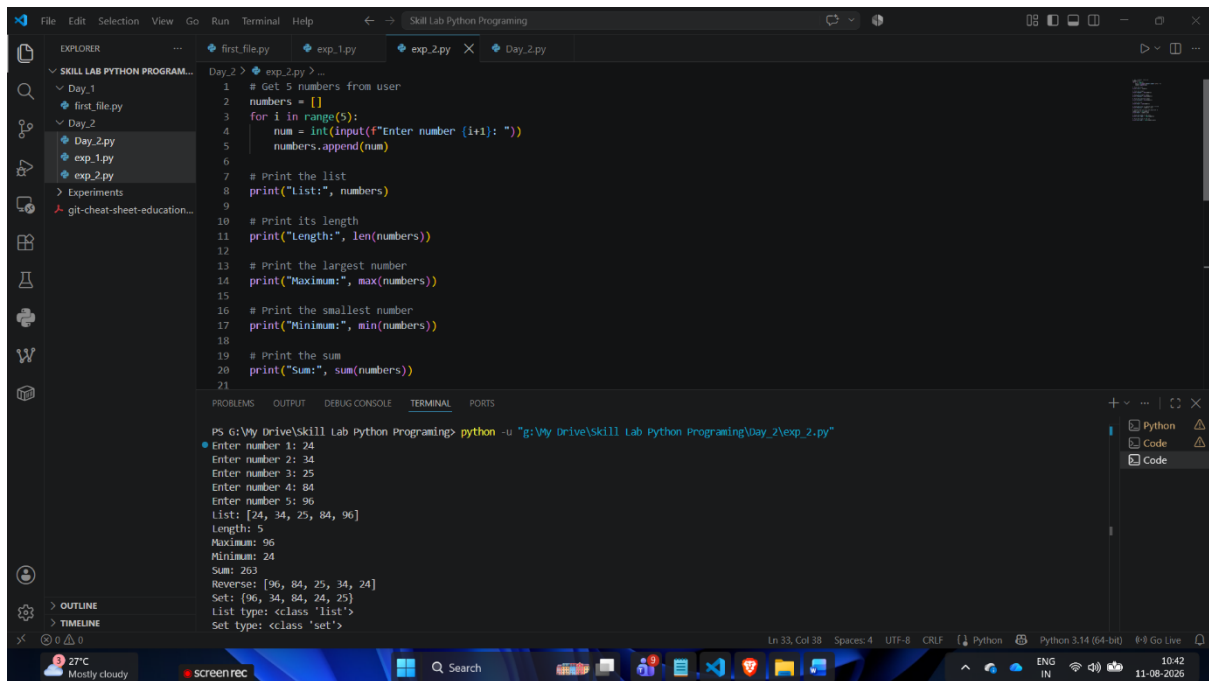
Tuple → immutable

Set → mutable + unique

String → immutable sequence of characters

The most important distinction is:

Mutability is about whether the existing object can be changed. Assigning a new value to a variable is not the same thing as modifying the existing object.



The screenshot displays a Visual Studio Code (VS Code) editor window titled "Skill Lab Python Programming". The Explorer sidebar on the left shows a project structure with folders "Day_1" and "Day_2", and files "first_file.py", "exp_1.py", and "exp_2.py". The "Day_2" folder is expanded, and "exp_2.py" is selected. The editor shows the following Python code:

```
1 # Get 5 numbers from user
2 numbers = []
3 for i in range(5):
4     num = int(input(f"Enter number {i+1}: "))
5     numbers.append(num)
6
7 # Print the list
8 print("List:", numbers)
9
10 # Print its length
11 print("Length:", len(numbers))
12
13 # Print the largest number
14 print("Maximum:", max(numbers))
15
16 # Print the smallest number
17 print("Minimum:", min(numbers))
18
19 # Print the sum
20 print("Sum:", sum(numbers))
21
```

The TERMINAL panel at the bottom shows the command prompt output for running the script:

```
PS G:\My Drive\Skill Lab Python Programing> python -u "g:\My Drive\Skill Lab Python Programing\Day_2\exp_2.py"
Enter number 1: 24
Enter number 2: 34
Enter number 3: 25
Enter number 4: 84
Enter number 5: 96
List: [24, 34, 25, 84, 96]
Length: 5
Maximum: 96
Minimum: 24
Sum: 263
Reverse: [96, 84, 25, 34, 24]
Set: {96, 34, 84, 24, 25}
List type: <class 'list'>
Set type: <class 'set'>
```

The status bar at the bottom indicates the current line and column as "Ln 33, Col 38", the encoding as "UTF-8", and the file type as "Python". The system tray shows the date and time as "10:42 11-08-2025".